

LAPORAN PRAKTIKUM 9
TREE DAN GRAPH TRAVERSAL
STRUKTUR DATA



Disusun Oleh:

Nama: Amiratul Fadhilah

NIM: 2511532023

Dosen Pengampu: Dr. Ir. Wahyudi, S. T., M. T

DEPARTEMEN INFORMATIKA
FAKULTAS TEKNOLOGI INFORMASI
UNIVERSITAS ANDALAS
PADANG

2026

KATA PENGANTAR

Puji syukur ke hadirat Allah Yang Maha Esa atas rahmat dan karunia-Nya, sehingga penulisan laporan praktikum dengan judul “Tree dan Graph Traversal” ini dapat diselesaikan tepat waktu. Laporan ini disusun sebagai salah satu syarat untuk memenuhi tugas mata kuliah Praktikum Struktur Data.

Laporan ini akan membahas secara mendalam mengenai konsep, logika dasar, serta implementasi struktur data non-linear, khususnya Tree dan Graph, menggunakan bahasa pemrograman Java. Melalui pembahasan ini, diharapkan pembaca dapat memahami bagaimana data disimpan secara hierarkis maupun dalam bentuk jaringan node yang saling terhubung, serta bagaimana menelusurinya menggunakan berbagai algoritma *traversal*.

Penulis menyadari bahwa masih terdapat berbagai kekurangan, baik dari segi penulisan maupun analisis kode program. Oleh karena itu, kritik dan saran yang membangun dari dosen pengampu maupun asisten praktikum sangat diharapkan demi perbaikan di masa mendatang. Semoga dokumentasi praktikum ini dapat memberikan manfaat tambahan bagi pembaca.

Padang, 9 Juni 2026

Amiratul Fadhillah

DAFTAR ISI

KATA PENGANTAR.....	i
DAFTAR ISI	ii
BAB I PENDAHULUAN	
1.1 Latar Belakang.....	1
1.2 Tujuan Praktikum	1
1.3 Manfaat Praktikum	2
BAB II PEMBAHASAN	
2.1 Landasan Teori	3
2.2 Alat dan Bahan.....	3
2.3 Langkah Praktikum dan Kode Program	4
2.3.1 Kelas Node	4
2.3.2 Kelas BTree	5
2.3.3 Kelas Btree Driver	6
2.3.4 Kelas Graph Traversal.....	7
BAB III PENUTUP	
3.1 Kesimpulan.....	13
3.2 Saran.....	13
DAFTAR PUSTAKA.....	14
TUGAS 8 PRAKTIKUM STRUKTUR DATA.....	15

BAB I

PENDAHULUAN

1.1 Latar Belakang

Dalam dunia ilmu komputer, struktur data memegang peranan vital dalam menentukan efisiensi dan kerangka arsitektur dari sebuah program. Setelah menguasai struktur data linear sederhana seperti Array dan Linked List, pemahaman terhadap struktur data non-linear menjadi lompatan selanjutnya yang sangat krusial. Struktur data linear memiliki keterbatasan dan seringkali tidak cukup fleksibel untuk memodelkan hubungan informasi yang kompleks, bercabang, atau memiliki banyak relasi silang secara alami. Oleh karena itu, diperkenalkanlah struktur dinamis Tree (Pohon) dan Graph (Graf) yang mampu merepresentasikan data di dunia nyata secara jauh lebih logis dan terstruktur.

Tree sangat efektif diaplikasikan untuk menggambarkan tipe data yang memiliki tingkatan atau hierarki kuat, seperti struktur susunan direktori komputer, logika permainan, atau silsilah sebuah objek. Di sisi lain, Graph hadir untuk memodelkan jaringan nyata yang tidak selalu berakar di satu titik pusat, seperti rute transportasi darat, topologi jaringan komputer, atau relasi pertemanan di media sosial. Kedua struktur kompleks ini membutuhkan metode penelusuran (*traversal*) khusus agar program komputer dapat mengunjungi dan mengakses setiap elemen datanya dengan tepat. Pemahaman akan algoritma seperti *Depth First Search* (DFS) dan *Breadth First Search* (BFS) merupakan teknik fundamental yang harus dikuasai oleh setiap akademisi informatika untuk melakukan navigasi data secara mutlak dan terencana.

1.2 Tujuan Praktikum

Tujuan dari pelaksanaan praktikum struktur data mengenai *Tree dan Graph Traversal* ini adalah sebagai berikut:

- 1) Memberikan pemahaman teoretis serta praktis kepada mahasiswa mengenai kerangka struktur data non-linear.
- 2) Melatih kemampuan mahasiswa dalam merancang dan memanipulasi *Binary Tree* (Pohon Biner) beserta eksekusi tiga metode *traversal* utama (*Inorder, Preorder, Postorder*) menggunakan instruksi Java.

- 3) Membekali mahasiswa dengan keterampilan menyusun rancangan *Graph* di dalam memori menggunakan struktur dasar *Adjacency List*.
- 4) Mempraktikkan dan mengamati perbedaan performa penelusuran *Graph* saat diuji menggunakan algoritma rekursif *Depth First Search* (DFS) dan iteratif *Breadth First Search* (BFS).

1.3 Manfaat Praktikum

Pelaksanaan praktikum ini diharapkan memberikan manfaat yang signifikan dan saling berkesinambungan baik secara akademis maupun teknis. Secara akademis, mahasiswa tidak lagi sekadar mendengarkan dan membayangkan wujud teori graf maupun pohon secara abstrak, melainkan dapat melihat eksekusi dan perpindahan logikanya secara nyata melalui hasil *output console*.

Secara teknis, kemampuan pemecahan masalah (*problem-solving*) mahasiswa akan terasah secara tajam. Dengan terbiasa memecahkan studi kasus penelusuran DFS maupun BFS, mahasiswa terlatih untuk berpikir sistematis dalam mengendalikan aliran pointer dan referensi objek yang rumit. Kemampuan logika *traversal* ini akan menjadi senjata dan pondasi kuat ketika kelak berhadapan dengan pengembangan perangkat lunak tingkat tinggi di masa yang akan datang.

BAB II

PEMBAHASAN

2.1 Landasan Teori

Tree (Pohon) adalah sebuah koleksi data non-linear yang strukturnya menyerupai sebuah pohon fisik yang dibalik. Ia terdiri dari serangkaian *node* (simpul) yang saling dikaitkan oleh petunjuk arah (*edge*) dengan mempertahankan hubungan hierarkis absolut layaknya induk dan anak. Bentuk paling presisi dari tipe ini adalah *Binary Tree*, di mana setiap simpul dibatasi untuk hanya memiliki maksimal dua buah anak cabang, yaitu anak cabang kiri (*left child*) dan anak cabang kanan (*right child*). Metode penelusurannya terbagi menjadi tiga urutan eksekusi, yakni *Preorder* (Pusat-Kiri-Kanan), *Inorder* (Kiri-Pusat-Kanan), dan *Postorder* (Kiri-Kanan-Pusat), yang menjadikannya struktur terdepan untuk urusan pengurutan.

Graph adalah kerangka induk yang lebih luwes jika dibandingkan dengan Tree. Graph terbentuk dari sekumpulan simpul (*vertices*) dan pita garis penghubung (*edges*). Berbeda jauh dengan konsep *Binary Tree*, kerangka Graph membuang aturan mengenai titik puncak berakar tunggal (*root*) dan memperbolehkan hadirnya sebuah putaran relasi memutar atau siklus (*cycle*). Untuk membaca peta pada graf, dikembangkan dua metode klasik yang diakui secara global. Metode pertama adalah *Depth First Search* (DFS), yang bergerak menyusup sedalam mungkin ke dalam dahan jaringan sebelum memutuskan untuk kembali mundur mencari cabang lainnya. Metode kedua adalah *Breadth First Search* (BFS), yang bekerja secara melebar, menelusuri secara sabar lapisan jaringan terdekat lapis demi lapis dengan bantuan antrean (*Queue*).

2.2 Alat dan Bahan

- 1) Perangkat keras (Hardware): Komputer atau Laptop.
- 2) Sistem Operasi (OS): Windows, macOS, atau distribusi Linux.
- 3) Perangkat Lunak (Software): Java Development Kit (JDK), serta Integrated Development Environment (IDE) seperti Eclipse.

2.3 Langkah Praktikum dan Kode Program

2.3.1 Kelas Node

Langkah awal dalam merancang *Binary Tree* adalah memastikan kita memiliki wadah penampung terkecil. Kode di bawah ini adalah cetak biru untuk menciptakan simpul (node) mandiri.

```
package pekan9_2511532023;

public class Node_2511532023 {
    int data_2023;
    Node_2511532023 left_2023;
    Node_2511532023 right_2023;
    public Node_2511532023(int data_2023) {
        this.data_2023 = data_2023;
        left_2023 = null;
        right_2023 = null;
    }
    public void setLeft_2023(Node_2511532023 node_2023) {
        if (left_2023 == null)
            left_2023 = node_2023;
    }
    public void setRight_2023(Node_2511532023 node_2023) {
        if (right_2023 == null)
            right_2023 = node_2023;
    }
    public Node_2511532023 getLeft_2023() {
        return left_2023;
    }
    public Node_2511532023 getRight_2023() {
        return right_2023;
    }
    public int getData_2023() {
        return data_2023;
    }
    public void setData_2023(int data_2023) {
        this.data_2023 = data_2023;
    }

    void printPreorder_2023(Node_2511532023 node_2023) {
        if (node_2023 == null)
            return;
        System.out.print(node_2023.data_2023 + " ");
        printPreorder_2023(node_2023.left_2023);
        printPreorder_2023(node_2023.right_2023);
    }
    void printPostorder_2023(Node_2511532023 node_2023) {
        if (node_2023 == null)
            return;
        printPostorder_2023(node_2023.left_2023);
        printPostorder_2023(node_2023.right_2023);
        System.out.print(node_2023.data_2023 + " ");
    }
    void printInorder_2023(Node_2511532023 node_2023) {
        if (node_2023 == null)
            return;
        printInorder_2023(node_2023.left_2023);
        System.out.print(node_2023.data_2023 + " ");
    }
}
```

```

        printInorder_2023(node_2023.right_2023);
    }
    public String print_2023() {
        return this.print_2023("", true, "");
    }
    public String print_2023(String prefix_2023, boolean isTail_2023,
String sb_2023) {
        if (right_2023 != null) {
            right_2023.print_2023(prefix_2023 + (isTail_2023 ?
"|   " : "   "), false, sb_2023);
        }
        System.out.println(prefix_2023 + (isTail_2023 ? "\\-- " :
"/-- ") + data_2023);
        if (left_2023 != null) {
            left_2023.print_2023(prefix_2023 + (isTail_2023 ? "
: "|   "), true, sb_2023);
        }
        return sb_2023;
    }
}

```

Analisis Kode:

- int data_2023, left_2023, dan right_2023: Merupakan instrumen pembentuk *node*. Nilai angkanya disimpan utuh ke dalam variabel data, sedangkan referensi ke tetangganya dipandu oleh pointer objek left dan right.
- public Node_2511532023(int data_2023): Merupakan konstruktor. Ketika wadah baru terbentuk, nilai angkanya langsung diikat, sementara jalur penunjuk kiri maupun kanannya diatur kosong (null) terlebih dahulu sebelum dikaitkan.
- printPreorder_2023: Merupakan otak dari fungsi cetak rekursif. Fungsi ini didesain cerdas untuk mengecek isi simpul induk, mengeksekusi (mencetak) isinya, sebelum melempar kembali fungsi dirinya sendiri secara mandiri ke kedalaman sebelah kiri dan lalu mengulangnya lagi ke arah sebelah kanan.

2.3.2 Kelas BTree

Tugas kelas ini adalah memastikan kumpulan *node* bebas diikat secara komprehensif ke dalam hierarki sistem organisasi *Binary Tree*.

```

package pekan9_2511532023;

public class BTree_2511532023 {
    private Node_2511532023 root_2023;

```



```

private Node_2511532023 currentNode_2023;
public BTree_2511532023() {
    root_2023 = null;
}
public boolean search_2023(int data_2023) {
    return search_2023(root_2023, data_2023);
}
private boolean search_2023(Node_2511532023 node_2023, int
data_2023) {
    if (node_2023.getData_2023() == data_2023)
        return true;
    if (node_2023.getLeft_2023() != null)
        if (search_2023(node_2023.getLeft_2023(),
data_2023))
            return true;
    if (node_2023.getRight_2023() != null)
        if (search_2023(node_2023.getRight_2023(),
data_2023))
            return true;
    return false;
}
public void printInorder_2023() {
    root_2023.printInorder_2023(root_2023);
}
public void printPreorder_2023() {
    root_2023.printPreorder_2023(root_2023);
}
public void printPostorder_2023() {
    root_2023.printPostorder_2023(root_2023);
}

public Node_2511532023 getRoot_2023() {
    return root_2023;
}

public boolean isEmpty_2023() {
    return root_2023 == null;
}
public int countNodes_2023() {
    return countNodes_2023(root_2023);
}

private int countNodes_2023(Node_2511532023 node_2023) {
    int count_2023 = 1;
    if (node_2023 == null) {
        return 0;
    } else {
        count_2023 +=
countNodes_2023(node_2023.getLeft_2023());
        count_2023 +=
countNodes_2023(node_2023.getRight_2023());
        return count_2023;
    }
}

public void print_2023() {
    root_2023.print_2023();
}

```

```

    }

    public Node_2511532023 getCurrent_2023() {
        return currentNode_2023;
    }

    public void setCurrent(Node_2511532023 node_2023) {
        this.currentNode_2023 = node_2023;
    }

    public void setRoot_2023(Node_2511532023 root_2023) {
        this.root_2023 = root_2023;
    }
}

```

Analisis Kode:

- private Node_2511532023 root_2023: Menjadi penjaga pintu utama atau takhta tertinggi (titik akar). Jika isi root bernilai null, dapat dipastikan struktur masih gersang dan belum ada ranting yang tumbuh.
- search_2023: Melacak keberadaan nilai dari cabang ke cabang secara rekursif murni. Ketika angka target berhasil dijejaki, perburuan dihentikan dan bendera kelulusan dengan status kembalian tipe *boolean* berupa true segera dinaikkan.
- countNodes_2023: Memiliki kewajiban murni sebagai fungsi kalkulator simpul. Algoritma ini berjalan otomatis menjumlahkan nilai 1 miliknya dengan anak cabang kiri dan kanannya ke dalam sebuah rangkaian *return* bertahap yang presisi.

2.3.3 Kelas Btree Driver

Berperan penting dalam memanggil, menciptakan relasi secara langsung, dan mengeksekusi operasi tampilan di layar pengguna.

```

package pekan9_2511532023;

public class BtreeDriver_2511532023 {
    public static void main(String[] args) {
        // Membuat pohon
        BTree_2511532023 tree = new BTree_2511532023();
        System.out.print("Jumlah Simpul awal pohon: ");
        System.out.println(tree.countNodes_2023());
        // Menambahkan simpul data 1
        Node_2511532023 root = new Node_2511532023(1);
        // Menjadikan simpul 1 sebagai root
        tree.setRoot_2023(root);
        System.out.println("Jumlah simpul jika hanya ada root");
        System.out.println(tree.countNodes_2023());
    }
}

```

```

Node_2511532023 node2_2023 = new Node_2511532023(2);
Node_2511532023 node3_2023 = new Node_2511532023(3);
Node_2511532023 node4_2023 = new Node_2511532023(4);
Node_2511532023 node5_2023 = new Node_2511532023(5);
Node_2511532023 node6_2023 = new Node_2511532023(6);
Node_2511532023 node7_2023 = new Node_2511532023(7);
Node_2511532023 node8_2023 = new Node_2511532023(8);
Node_2511532023 node9_2023 = new Node_2511532023(9);
root.setLeft_2023(node2_2023);
node2_2023.setLeft_2023(node4_2023);
node2_2023.setRight_2023(node5_2023);
node4_2023.setRight_2023(node8_2023);
root.setRight_2023(node3_2023);
node3_2023.setLeft_2023(node6_2023);
node3_2023.setRight_2023(node7_2023);
node6_2023.setLeft_2023(node9_2023);

// Set root
tree.setCurrent(tree.getRoot_2023());
System.out.println("Menampilkan simpul terakhir: ");
System.out.println(tree.getCurrent_2023().getData_2023());
System.out.println("Jumlah simpul; setelah simpul 7
ditambahkan");
System.out.println(tree.countNodes_2023());
System.out.println("InOrder: ");
tree.printInorder_2023();
System.out.println("\nPreorder: ");
tree.printPreorder_2023();
System.out.println("\nPostorder: ");
tree.printPostorder_2023();
System.out.println("\nDmenampilkan simpul dalam bentuk
pohon");
tree.print_2023();
}
}

```

Analisis Kode:

- BTree_2511532023 tree = new BTree_2511532023();: Menginisialisasi lahan utama objek manajer *Tree*.
- root.setLeft_2023(node2_2023);: Melakukan prosedur pemasangan jalur relasi. Simpul bernilai 2 diletakkan dengan presisi di arah cabang sebelah kiri dari puncak root. Teknik ini diterapkan terus menerus hingga diagram utuh pohon tercapai.
- tree.printInorder_2023();: Berfungsi sebagai pelatuk peluncur fungsi *traversal* yang sebelumnya telah ditanam dan dipersiapkan dalam kelas pengelola.

Output:

```

Console X
<terminated> BtreeDriver_2511532023 [Java Application] C:\Users\ACER\.p2\po
Jumlah Simpul awal pohon: 0
Jumlah simpul jika hanya ada root
1
Menampilkan simpul terakhir:
1
Jumlah simpul; setelah simpul 7 ditambahkan
9
InOrder:
4 8 2 5 1 9 6 3 7
Preorder:
1 2 4 8 5 3 6 9 7
Postorder:
8 4 5 2 9 6 7 3 1
Dmenampilkan simpul dalam bentuk pohon
|      /-- 7
|    /-- 3
|  |  \-- 6
|  |    \-- 9
|-- 1
|   /-- 5
|-- 2
|   /-- 8
|   \-- 4

```

2.3.4 Kelas Graph Traversal

Simulasi jalinan jaringan tanpa arah tunggal yang diperkuat dengan implementasi DFS dan BFS yang elegan.

```

package pekan9_2511532023;
import java.util.*;

public class GraphTraversal_2511532023 {
    private Map<String, List<String>> graph_2023 = new HashMap<>();

    // Menambahkan edge (graf tak berarah)
    public void addEdge_2023(String node1_2023, String node2_2023) {
        graph_2023.putIfAbsent(node1_2023, new ArrayList<>());
        graph_2023.putIfAbsent(node2_2023, new ArrayList<>());
        graph_2023.get(node1_2023).add(node2_2023);
        graph_2023.get(node2_2023).add(node1_2023);
    }

    // Menampilkan graf awal
    public void printGraph_2023() {
        System.out.println("Graf Awal (Adjacency List):");
        for (String node_2023 : graph_2023.keySet()) {
            System.out.print(node_2023 + " -> ");
            List<String> neighbors_2023 =
graph_2023.get(node_2023);

```

```

        System.out.println(String.join(", ",
neighbors_2023));
    }
    System.out.println();
}

// DFS rekursif
public void dfs_2023(String start_2023) {
    Set<String> visited_2023 = new HashSet<>();
    System.out.println("Penelusuran DFS: ");
    dfsHelper_2023(start_2023, visited_2023);
    System.out.println();
}

private void dfsHelper_2023(String current_2023, Set<String>
visited_2023) {
    if (visited_2023.contains(current_2023)) return;
    visited_2023.add(current_2023);
    System.out.print(current_2023 + " ");
    for (String neighbor_2023 :
graph_2023.getDefault(current_2023, new ArrayList<>())) {
        dfsHelper_2023(neighbor_2023, visited_2023);
    }
}

// BFS iteratif
public void bfs_2023(String start) {
    Set<String> visited_2023 = new HashSet<>();
    Queue<String> queue_2023 = new LinkedList<>();
    queue_2023.add(start);
    visited_2023.add(start);
    System.out.println("Penelusuran BFS: ");
    while (!queue_2023.isEmpty()) {
        String current_2023 = queue_2023.poll();
        System.out.print(current_2023 + " ");
        for (String neighbor_2023 :
graph_2023.getDefault(current_2023, new ArrayList<>())) {
            if (!visited_2023.contains(neighbor_2023)) {
                queue_2023.add(neighbor_2023);
                visited_2023.add(neighbor_2023);
            }
        }
    }
    System.out.println();
}

// Main
public static void main(String[] args) {
    GraphTraversal_2511532023 graph_2023 = new
GraphTraversal_2511532023();

    // Contoh graf: A-B, A-C, B-D, B-E
    graph_2023.addEdge_2023("A", "B");
    graph_2023.addEdge_2023("A", "C");
    graph_2023.addEdge_2023("B", "D");
    graph_2023.addEdge_2023("B", "E");
    // Cetak graf awal
    System.out.println("Graf Awal adalah: ");
    graph_2023.printGraph_2023();
    // Lakukan penelusuran

```

```
graph_2023.dfs_2023("A");
graph_2023.bfs_2023("A");
    }
}
```

Analisis Kode:

- `Map<String, List<String>> graph_2023`: Memanfaatkan pustaka modern Java Collection (*HashMap*) untuk menciptakan pola *Adjacency List*. Pemetaan ini berfungsi mendaftarkan nama simpul (*String*) agar berpasangan akurat dengan koleksi nama tetangganya di dalam format *List*.
- `addEdge_2023`: Memastikan pemasangan *edge* dilakukan secara simetris dan dua arah (*bidirectional*). Ketika pintu simpul pertama mengarah ke simpul kedua, relasi pantulan dari simpul kedua ke simpul pertama juga diamankan.
- `dfsHelper_2023`: Metode DFS berbasis rekursif murni. Pengamanan mutlak disertakan melalui implementasi objek *Set visited*. Tanpanya, jaringan yang terhubung memutar (*cycle*) akan melempar kode program ke dalam dimensi kegagalan *infinite loop*.
- `bfs_2023`: Melancarkan invasi pembacaan memori graf dengan bantuan lapisan antrean (*Queue*). Secara iteratif, tetangga-tetangga yang mengitari root mula-mula disiapkan secara FIFO (*First-In-First-Out*) hingga semua zona tetangga terjelajahi selapis demi selapis.

Output:

```
Console X
<terminated> GraphTraversal_2511532023 [Java]
Graf Awal adalah:
Graf Awal (Adjacency List):
A -> B, C
B -> A, D, E
C -> A
D -> B
E -> B

Penelusuran DFS:
A B D E C
Penelusuran BFS:
A B C D E
```

BAB III

PENUTUP

3.1 Kesimpulan

Berdasarkan praktikum komputasi terstruktur yang telah dilaksanakan, dapat disimpulkan bahwa secara pondasi *pointer*, objek Tree dan Graph memiliki kesamaan konseptual, namun memiliki peruntukan fungsional yang jauh berbeda. *Binary Tree* membentuk ekosistem hierarki logis dengan tingkat kedisiplinan yang tinggi (anak terbatas), menjadikannya instrumen unggulan dalam proses optimasi pencarian data biner, pembentukan silsilah pohon, hingga representasi pohon ekspresi matematis. Keharusan pergerakannya dipetakan rapi di dalam konsep tiga pergerakan linear (*Preorder*, *Inorder*, *Postorder*).

Sebaliknya, model data Graph mengekspresikan wujud jalinan koneksi terkuat dan paling natural dari dunia nyata yang sering memuat siklus acak. Modifikasi implementasi *Adjacency List* menggunakan instrumen dinamis Java terbukti menghemat alokasi ruang memori ketimbang menggunakan formasi *Matrix*. Selain itu, keberhasilan navigasi ke dalam arsitektur Graph sangat berutang pada kecerdasan manajemen jejak jejak kunjungan rekursif yang ada di DFS, serta stabilitas navigasi berlapis dari antrean iteratif pada metode BFS.

3.2 Saran

Di masa penyusunan kode arsitektur program yang lebih besar nantinya, sangat disarankan bagi para pengembang tingkat lanjut untuk lebih berhati-hati dalam menavigasi jebakan lubang batas rekursi tak berujung (kasus mutlak *StackOverflowError*). Selalu kawal setiap deklarasi perulangan DFS maupun *Tree Traversal* dengan kondisi pelepasan (*base condition*) yang aman dan valid, misalnya rutinitas proteksi pengecekan variabel null atau contains.

Penggambaran sketsa topologi silsilah *node* ke dalam medium kertas kasar dengan bantuan pena akan menghindarkan insinyur peranti lunak dari salah urat perujukan pointer *left* maupun *right*. Teknik analisis manual ini adalah rutinitas wajib dalam penelusuran relasi graf, untuk menjamin integritas tetangga yang terhubung tak pernah luput dikompilasi atau justru masuk ganda ke dalam antrean BFS.

DAFTAR PUSTAKA

- [1] R. Saiudin, "Laporan Praktikum Struktur Data Modul Tree," *Scribd*, 2024. [Daring]. Tersedia pada: <https://id.scribd.com/document/736925069/RAHMAT-SAIUDIN-2311103091-LAPORAN-PRAKTIKUM-STRUKTUR-DATA-MODUL-1X-TREE>.
- [2] N. Anonim, "Struktur Data Graph dan Implementasinya," *Scribd*, 2023. [Daring]. Tersedia pada: <https://id.scribd.com/document/634680405/Untitled>.
- [3] B. A. Mahendra, "MODUL 6 Graph - Struktur Data Graph untuk Jaringan Jalan," *Scribd*, 2022. [Daring]. Tersedia pada: <https://id.scribd.com/document/581065273/MODUL-6-Graph>.
- [4] N. Anonim, "Praktikum Struktur Data Tree dan Pointers," *Scribd*, 2023. [Daring]. Tersedia pada: <https://id.scribd.com/document/634326285/Laprak-Tree>.

TUGAS 9 PRAKTIKUM STRUKTUR DATA

Soal

Implementasikan algoritma BFS (Breadth First Search) dan DFS (Depth First Search) menggunakan bahasa Java dengan antarmuka GUI.

Studi kasus yang digunakan adalah pencarian jalur pada suatu peta lokasi yang direpresentasikan dalam bentuk Graph.

Setiap mahasiswa wajib membuat peta lokasi yang berbeda dengan mahasiswa lain. Peta dapat berupa: Gedung Perkuliahan Fakultas, Universitas, Sekolah, Rumah Sakit, Mall, Tempat Wisata, Bandara, atau lokasi lainnya.

Program harus mampu: Menampilkan graph dalam bentuk visual, Menentukan titik awal (Start), Menentukan titik tujuan (Goal), Melakukan pencarian menggunakan BFS, Melakukan pencarian menggunakan DFS, Menampilkan jalur yang ditemukan, Menampilkan urutan simpul yang dikunjungi, Urutan node yang dikunjungi, Jumlah node yang dieksplorasi.

Jawab

A. Kelas Peta Beijing

Secara garis besar, program ini adalah aplikasi desktop berbasis Java yang memvisualisasikan algoritma Breadth-First Search (BFS) dan Depth-First Search (DFS).

Program memodelkan peta 10 lokasi wisata terkenal di Beijing ke dalam bentuk Graph Tidak Berarah. Lokasi wisata bertindak sebagai simpul (node), sedangkan jalan penghubung antar lokasi bertindak sebagai sisi (edge). Graph ini disimpan menggunakan struktur data HashMap. Saat pengguna memilih lokasi awal dan tujuan lalu menekan tombol metode pencarian, program akan menelusuri graph secara visual, mewarnai node yang sedang dieksplorasi, dan menandai jalur akhir yang ditemukan, lalu mencetak rinciannya di area teks.

```
package pekan9_2511532023;  
  
import javax.swing.*.*;  
import java.awt.*.*;  
import java.util.*.*;
```

```

import java.util.List;

public class PetaBeijing_2511532023 extends JFrame {

    // Struktur Data Graph
    private Map<String, List<String>> graph_2023 = new HashMap<>();
    private Map<String, Point> koordinat_2023 = new HashMap<>();
    private Map<String, Color> warnaNode_2023 = new HashMap<>();

    // Komponen GUI
    private JComboBox<String> cbAwal_2023, cbTujuan_2023;
    private JTextArea taHasil_2023;
    private JPanel panelPeta_2023;

    // Nama lokasi Wisata Beijing
    private String[] daftarLokasi_2023 = {
        "Tiananmen Square", "Forbidden City", "National Museum",
        "Beijing Zoo",
        "Temple of Heaven", "Beihai Park", "Yonghe Temple", "Summer
        Palace",
        "Bird's Nest Stadium", "Great Wall of China"
    };

    public PetaBeijing_2511532023() {
        setTitle("Pencarian Jalur Peta Beijing (BFS & DFS)");
        setSize(850, 650);
        setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);
        setLayout(new BorderLayout());

        inisialisasiData_2023();
        buatGUI_2023();
    }

    private void inisialisasiData_2023() {
        for (String lokasi_2023 : daftarLokasi_2023) {
            graph_2023.put(lokasi_2023, new ArrayList<>());
            warnaNode_2023.put(lokasi_2023, Color.WHITE);
        }

        tambahJalur_2023("Tiananmen Square", "Forbidden City");
        tambahJalur_2023("Tiananmen Square", "National Museum");
        tambahJalur_2023("Tiananmen Square", "Beijing Zoo");
        tambahJalur_2023("Temple of Heaven", "Tiananmen Square");
        tambahJalur_2023("Temple of Heaven", "National Museum");
        tambahJalur_2023("National Museum", "Beijing Zoo");
        tambahJalur_2023("Forbidden City", "Beihai Park");
        tambahJalur_2023("Forbidden City", "Beijing Zoo");
        tambahJalur_2023("Beijing Zoo", "Yonghe Temple");
        tambahJalur_2023("Beihai Park", "Yonghe Temple");
        tambahJalur_2023("Beihai Park", "Summer Palace");
        tambahJalur_2023("Yonghe Temple", "Bird's Nest Stadium");
        tambahJalur_2023("Summer Palace", "Bird's Nest Stadium");
        tambahJalur_2023("Summer Palace", "Great Wall of China");
        tambahJalur_2023("Bird's Nest Stadium", "Great Wall of China");

        koordinat_2023.put("Great Wall of China", new Point(150, 50));
    }
}

```

```

        koordinat_2023.put("Summer Palace", new Point(250, 150));
        koordinat_2023.put("Bird's Nest Stadium", new Point(500, 100));
        koordinat_2023.put("Beihai Park", new Point(300, 250));
        koordinat_2023.put("Yonghe Temple", new Point(600, 200));
        koordinat_2023.put("Forbidden City", new Point(450, 300));
        koordinat_2023.put("Beijing Zoo", new Point(650, 350));
        koordinat_2023.put("Tiananmen Square", new Point(450, 400));
        koordinat_2023.put("National Museum", new Point(600, 450));
        koordinat_2023.put("Temple of Heaven", new Point(450, 500));
    }

    private void tambahJalur_2023(String asal_2023, String tujuan_2023)
    {
        graph_2023.get(asal_2023).add(tujuan_2023);
        graph_2023.get(tujuan_2023).add(asal_2023);
    }

    private void buatGUI_2023() {
        JPanel panelAtas_2023 = new JPanel();
        panelAtas_2023.setBackground(Color.LIGHT_GRAY);

        cbAwal_2023 = new JComboBox<>(daftarLokasi_2023);
        cbTujuan_2023 = new JComboBox<>(daftarLokasi_2023);
        cbTujuan_2023.setSelectedIndex(9);

        JButton btnBFS_2023 = new JButton("BFS");
        JButton btnDFS_2023 = new JButton("DFS");
        JButton btnReset_2023 = new JButton("Reset");

        panelAtas_2023.add(new JLabel("Awal:"));
        panelAtas_2023.add(cbAwal_2023);
        panelAtas_2023.add(new JLabel("Tujuan:"));
        panelAtas_2023.add(cbTujuan_2023);
        panelAtas_2023.add(btnBFS_2023);
        panelAtas_2023.add(btnDFS_2023);
        panelAtas_2023.add(btnReset_2023);

        panelPeta_2023 = new JPanel() {
            @Override
            protected void paintComponent(Graphics g_2023) {
                super.paintComponent(g_2023);
                displayGraph(g_2023);
            }
        };
        panelPeta_2023.setBackground(new Color(245, 245, 235));

        taHasil_2023 = new JTextArea(6, 50);
        taHasil_2023.setEditable(false);
        taHasil_2023.setFont(new Font("Monospaced", Font.PLAIN, 14));
        taHasil_2023.setText("Hasil Pencarian:\nSilakan pilih lokasi dan
metode pencarian.");
        JScrollPane scrollPanel_2023 = new JScrollPane(taHasil_2023);

        add(panelAtas_2023, BorderLayout.NORTH);
        add(panelPeta_2023, BorderLayout.CENTER);
        add(scrollPanel_2023, BorderLayout.SOUTH);
    }

```

```

        btnBFS_2023.addActionListener(e ->
jalankanPencarian_2023("BFS"));
        btnDFS_2023.addActionListener(e ->
jalankanPencarian_2023("DFS"));
        btnReset_2023.addActionListener(e -> resetGraph());
    }

    public void displayGraph(Graphics g_2023) {
        g_2023.setFont(new Font("Arial", Font.BOLD, 12));

        g_2023.setColor(Color.BLACK);
        for (String lokasi_2023 : graph_2023.keySet()) {
            Point p1_2023 = koordinat_2023.get(lokasi_2023);
            for (String tetangga_2023 : graph_2023.get(lokasi_2023)) {
                Point p2_2023 = koordinat_2023.get(tetangga_2023);
                g_2023.drawLine(p1_2023.x, p1_2023.y, p2_2023.x,
p2_2023.y);
            }
        }

        int r_2023 = 15;
        for (String lokasi_2023 : daftarLokasi_2023) {
            Point p_2023 = koordinat_2023.get(lokasi_2023);

            g_2023.setColor(warnaNode_2023.get(lokasi_2023));
            g_2023.fillOval(p_2023.x - r_2023, p_2023.y - r_2023, r_2023
* 2, r_2023 * 2);

            g_2023.setColor(Color.BLACK);
            g_2023.drawOval(p_2023.x - r_2023, p_2023.y - r_2023, r_2023
* 2, r_2023 * 2);

            g_2023.drawString(lokasi_2023, p_2023.x - 30, p_2023.y +
30);
        }
    }

    public void resetGraph() {
        for (String lokasi_2023 : daftarLokasi_2023) {
            warnaNode_2023.put(lokasi_2023, Color.WHITE);
        }
        taHasil_2023.setText("Hasil Pencarian:\nMap di-reset.");
        panelPeta_2023.repaint();
    }

    private void jalankanPencarian_2023(String metode_2023) {
        resetGraph();
        String awal_2023 = (String) cbAwal_2023.getSelectedItem();
        String tujuan_2023 = (String) cbTujuan_2023.getSelectedItem();

        List<String> nodeDikunjungi_2023 = new ArrayList<>();
        Map<String, String> asalJalur_2023 = new HashMap<>();
        boolean ketemu_2023 = false;

        if (metode_2023.equals("BFS")) {
            ketemu_2023 = BFS(awal_2023, tujuan_2023,
nodeDikunjungi_2023, asalJalur_2023);

```

```

        } else {
            ketemu_2023 = DFS(awal_2023, tujuan_2023,
nodeDikunjungi_2023, asalJalur_2023);
        }

        displayPath(metode_2023, awal_2023, tujuan_2023,
nodeDikunjungi_2023, asalJalur_2023, ketemu_2023);
        panelPeta_2023.repaint();
    }

    public boolean BFS(String start_2023, String goal_2023, List<String>
dikunjungi_2023, Map<String, String> asal_2023) {
        Queue<String> antrean_2023 = new LinkedList<>();
        Set<String> sudahMasuk_2023 = new HashSet<>();

        antrean_2023.add(start_2023);
        sudahMasuk_2023.add(start_2023);

        while (!antrean_2023.isEmpty()) {
            String saatIni_2023 = antrean_2023.poll();
            dikunjungi_2023.add(saatIni_2023);
            warnaNode_2023.put(saatIni_2023, Color.YELLOW);

            if (saatIni_2023.equals(goal_2023)) {
                return true;
            }

            for (String tetangga_2023 : graph_2023.get(saatIni_2023)) {
                if (!sudahMasuk_2023.contains(tetangga_2023)) {
                    sudahMasuk_2023.add(tetangga_2023);
                    asal_2023.put(tetangga_2023, saatIni_2023);
                    antrean_2023.add(tetangga_2023);
                }
            }
        }
        return false;
    }

    public boolean DFS(String start_2023, String goal_2023, List<String>
dikunjungi_2023, Map<String, String> asal_2023) {
        Stack<String> tumpukan_2023 = new Stack<>();
        Set<String> sudahMasuk_2023 = new HashSet<>();

        tumpukan_2023.push(start_2023);

        while (!tumpukan_2023.isEmpty()) {
            String saatIni_2023 = tumpukan_2023.pop();

            if (!sudahMasuk_2023.contains(saatIni_2023)) {
                sudahMasuk_2023.add(saatIni_2023);
                dikunjungi_2023.add(saatIni_2023);
                warnaNode_2023.put(saatIni_2023, Color.ORANGE);

                if (saatIni_2023.equals(goal_2023)) {
                    return true;
                }
            }
        }
    }

```

```

        for (String tetangga_2023 :
graph_2023.get(saatIni_2023)) {
            if (!sudahMasuk_2023.contains(tetangga_2023)) {
                asal_2023.put(tetangga_2023, saatIni_2023);
                tumpukan_2023.push(tetangga_2023);
            }
        }
    }
}
return false;
}

public void displayPath(String metode_2023, String awal_2023, String
tujuan_2023, List<String> dikunjungi_2023, Map<String, String>
asal_2023, boolean ketemu_2023) {
    StringBuilder teks_2023 = new StringBuilder();
    teks_2023.append("Metode: ").append(metode_2023).append("\n");

    if (ketemu_2023) {
        List<String> rute_2023 = new ArrayList<>();
        String step_2023 = tujuan_2023;
        while (step_2023 != null && !step_2023.equals(awal_2023)) {
            rute_2023.add(step_2023);
            step_2023 = asal_2023.get(step_2023);
        }
        rute_2023.add(awal_2023);
        Collections.reverse(rute_2023);

        for (String lokasiRute_2023 : rute_2023) {
            warnaNode_2023.put(lokasiRute_2023, Color.GREEN);
        }

        teks_2023.append("Jalur Ditemukan : ").append(String.join("
-> ", rute_2023)).append("\n");
    } else {
        teks_2023.append("Jalur TIDAK Ditemukan!\n");
    }

    teks_2023.append("Node Dikunjungi : ").append(String.join(", ",
dikunjungi_2023)).append("\n");
    teks_2023.append("Jumlah Node Dieksplorasi :
").append(dikunjungi_2023.size());

    taHasil_2023.setText(teks_2023.toString());
}

public static void main(String[] args_2023) {
    SwingUtilities.invokeLater(() -> {
        new PetaBeijing_2511532023().setVisible(true);
    });
}
}

```

Analisis Kode:

a. Struktur Data Graph

- `graph_2023`: Sebuah HashMap yang menyimpan Adjacency List. Kunci (key) adalah nama lokasi, dan nilainya (value) adalah daftar lokasi tetangga yang terhubung langsung dengannya.
 - `koordinat_2023`: Menyimpan titik koordinat (X dan Y) untuk setiap lokasi agar bisa digambar secara presisi.
 - `warnaNode_2023`: Menyimpan status warna setiap node (Putih = belum dikunjungi, Kuning = dieksplorasi BFS, Oranye = dieksplorasi DFS, Hijau = bagian dari jalur akhir).
- b. Method `inisialisasiData_2023()` & `tambahJalur_2023()`
- Method ini bertugas membangun graph sesuai syarat tugas (minimal 10 node dan 15 edge).
- Di sini, 10 lokasi wisata dimasukkan ke dalam graph.
 - `tambahJalur_2023()` dipanggil 15 kali untuk menciptakan 15 edge. Karena ini undirected graph, setiap kali jalur A ke B ditambahkan, jalur B ke A juga otomatis ditambahkan.
 - Titik koordinat masing-masing lokasi juga ditetapkan di method ini.
- c. Method `buatGUI_2023()` & `displayGraph()`
- `buatGUI_2023()`: Membangun antarmuka pengguna, termasuk panel atas (berisi dropdown pilihan dan tombol-tombol), panel tengah (kanvas untuk menggambar peta), dan area bawah (untuk teks hasil).
 - `displayGraph()`: Method khusus yang bertugas menggambar garis (edge) antar koordinat yang terhubung, lalu menggambar lingkaran (node) beserta warnanya, dan mencetak teks nama lokasi di sebelahnya. Method ini diwajibkan oleh soal.
- d. Method Algoritma: BFS()
- Method ini menjalankan algoritma *Breadth-First Search*.
- Menggunakan struktur data Queue (Antrean) dengan konsep First-In-First-Out (FIFO).
 - Algoritma ini menyebar ke semua tetangga terdekat secara merata (level demi level) sebelum melangkah lebih jauh.
 - Setiap node yang dieksplorasi dicatat, diwarnai kuning, dan lokasi asalnya disimpan dalam Map untuk melacak jalur pulang (backtracking) nanti.

e. Method Algoritma: DFS()

Method ini menjalankan algoritma *Depth-First Search*.

- Menggunakan struktur data Stack (Tumpukan) dengan konsep Last-In-First-Out (LIFO).
- Algoritma ini menelusuri satu jalur sedalam-dalamnya sampai mentok, sebelum mundur dan mencari cabang lain.
- Setiap node yang dieksplorasi dicatat, diwarnai oranye, dan lokasi asalnya juga disimpan untuk pelacakan jalur.

f. Method displayPath() & resetGraph()

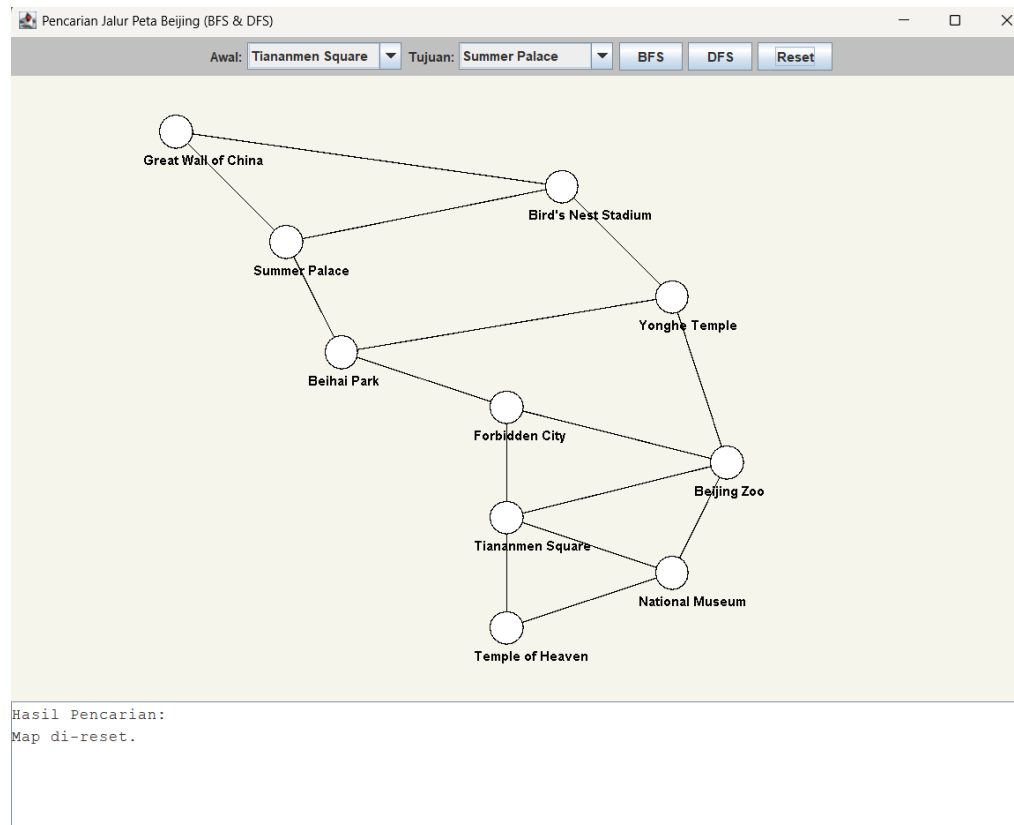
- displayPath(): Diwajibkan oleh soal. Method ini mengekstrak jalur dari tujuan kembali ke awal menggunakan data yang direkam selama BFS/DFS, lalu membalikinya agar urutannya benar (Awal -> Tujuan). Node yang masuk dalam rute ini diubah warnanya menjadi hijau, dan informasi seperti jumlah node dieksplorasi dicetak ke layar teks.
- resetGraph(): Mengembalikan semua warna node menjadi putih dan membersihkan teks agar map siap untuk pencarian baru.

Output:

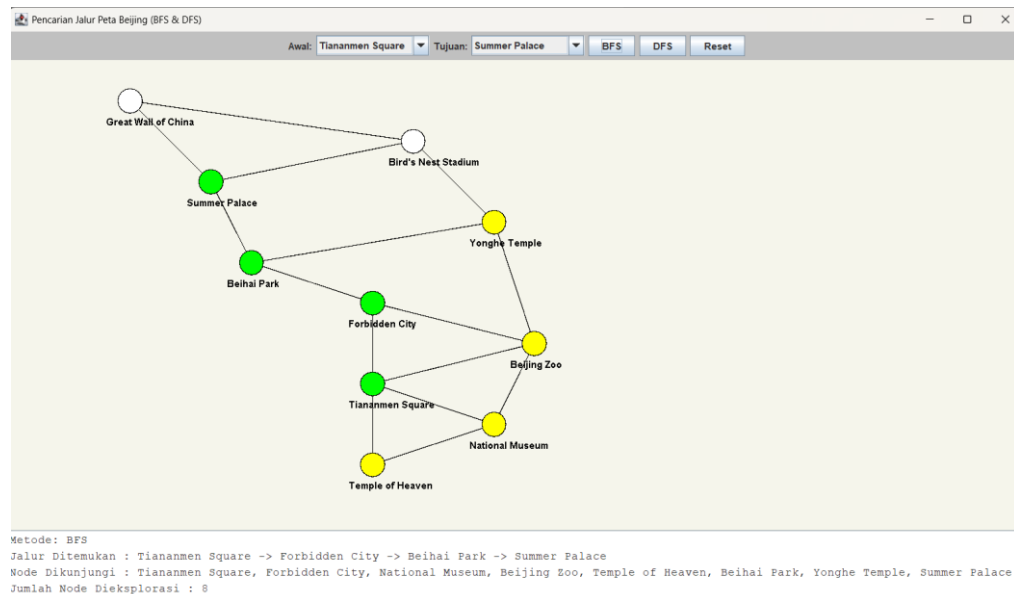
Titik awal: Tiananmen Square

Titik tujuan: Summer Palace

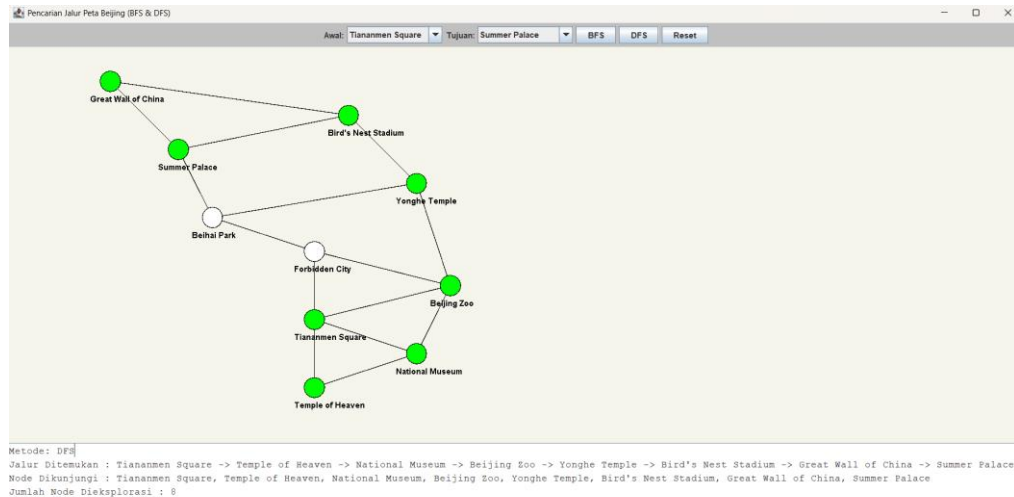
1) Kondisi awal



2) Setelah menjalankan BFS



3) Setelah menjalankan DFS



B. Kesimpulan

Berdasarkan pengujian rute dari Tiananmen Square ke Summer Palace, dapat disimpulkan bahwa algoritma Breadth-First Search (BFS) jauh lebih optimal dan efisien untuk kasus navigasi peta dibandingkan Depth-First Search (DFS). Meskipun kedua metode kebetulan mengeksplorasi jumlah node yang sama (8 node), BFS bekerja secara sistematis dengan mengecek area terdekat secara merata sehingga berhasil menjamin penemuan jalur terpendek yang hanya melewati 4 lokasi. Sebaliknya, DFS berfokus menelusuri satu cabang jalan sedalam-dalamnya hingga mentok, yang justru menghasilkan rute memutar dan tidak efisien dengan panjang 8 lokasi. Oleh karena itu, dalam studi kasus pencarian jalur tempuh tercepat pada sebuah peta lokasi, algoritma BFS merupakan pilihan yang paling tepat dan logis untuk digunakan.